

Exposé zur Bachelorthesis

Generative KI als Code-Analyse-Tool: Entwicklung und Evaluation eines LLM-Tools zur Erkennung von Qualitätsmängeln und Sicherheitslücken in Softwareprojekten.

Felix Krauß

2026-10-08

1 Zentrale Forschungsfrage

Inwieweit kann ein regelbasiertes, LLM-basiertes Analyse-Tool Qualitätsmängel und Sicherheitslücken in Quellcode zuverlässig erkennen, und wie schneidet es im Vergleich zu bestehenden Lösungen ab?

2 Zielsetzung der Arbeit

1. Überblick über den aktuellen Stand der Wissenschaft zur Verwendung generativer KI-Assistenten als Code-Analyse-Tools.

Fragen, die dazu beantwortet werden sollen:

Wie gut eignen sich laut dem derzeitigen Forschungsstand generative KI-Assistenten zum Erkennen von Qualitätsmängeln und Sicherheitslücken?

Welche Standards und Kriterien eignen sich, um die Qualität von Software zu bewerten?

Anhand welcher Kriterien und Klassifikationen werden Sicherheitslücken eingestuft?

Was sagt der derzeitige Forschungsstand zur Effizienz von KI-Assistenten beim Analysieren von Code, und welche Modelle eignen sich für diese Aufgabe?

2. Recherche, welche Lösungen es bereits gibt, die generative KI-Assistenten als Analyse-Tool verwenden, und was laut der Dokumentation deren Fähigkeiten und Limitationen sind. Die Ergebnisse der Recherche können als Unterstützung verwendet werden, um die Fähigkeiten des eigenen Tools zu definieren und um zu verstehen, welche Limitationen bei der Verwendung von generativen KI-Assistenten zur Analyse von Code existieren.
3. Entwicklung eines eigenen Tools zum Analysieren von Quellcode. Das Tool erhält den Code einer Anwendung als Input und erstellt auf Basis vordefinierter Regeln mithilfe eines LLMs einen Bericht, der die Qualität und Sicherheit des Codes bewertet und Handlungsvorschläge zur Verbesserung gibt.

Anforderungen an das Tool:

- Es wird ein Regelwerk definiert, das genau vorgibt, wie Qualität und Sicherheit des Codes gemessen werden. Das Tool orientiert seine Auswertung an diesen Regeln.
 - Die Regeln sollen so definiert sein, dass diese nicht auf bestimmte Anwendungen, Frameworks oder Programmiersprachen begrenzt sind. Idealerweise sollen die Regeln auf den Code aller möglichen Anwendungen, unabhängig vom Framework oder der verwendeten Programmiersprache, anwendbar sein.
 - Das Tool soll lokal deploybar sein und unabhängig von dem verwendeten LLM funktionieren. Das heißt, das verwendete LLM soll austauschbar sein, und es soll möglich sein, sowohl ein Cloud-Modell eines Anbieters als auch ein selbst betriebenes LLM zu verwenden.
4. Test des eigenen Tools anhand mehrerer Beispiele, die bekannte Sicherheitslücken und Qualitätsmängel enthalten, unter Verwendung verschiedener LLMs. Anschließend werden die Ergebnisse ausgewertet und mit vorher definierten KPIs abgeglichen, z. B.: Mindestens 95 % der bekannten Qualitätsmängel werden entdeckt, alle als kritisch eingestuft Sicherheitslücken werden erkannt und maximal 10 % der Meldungen sind Fehllalarme. Derselbe Test wird mit anderen Analyse-Tools durchgeführt, deren Ergebnisse nach denselben Kriterien ausgewertet und mit den Ergebnissen des eigenen Tools verglichen werden.
 5. (Optional) Einführung des Tools in die Infrastruktur der Hochschule, Dokumentation und eventuell Schulung zur Verwendung des Tools.

3 Aktueller Stand der Forschung

4 Vorläufige Gliederung

1. erstes Kapitel
2. zweites Kapitel